

Tugas 11: Praktikum Mandiri 11

Fatih Dzakwan Susilo - 0110224235

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: fatihdzakwansusilo@gmail.com

1. Praktikum Mandiri

Link Github : https://github.com/FatihDzkwn/Project_Machine_Learning.git

Link Dataset : <https://www.kaggle.com/datasets/mchirico/montcoalert>

1.1 Instalasi dan Import Library

```
[216]
✓ 8 d
!pip install folium
import folium
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.axes_grid1 import make_axes_locatable
from folium.plugins import HeatMap
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

...
Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from folium) (0.8.2)
Requirement already satisfied: Jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium) (3.1.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from folium) (2.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.4)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2025.11.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=2.9->folium) (3.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2025.11.12)
```

Gambar 1. Instalasi dan Import Library.

Kode ini menunjukkan proses instalasi library Folium menggunakan pip, diikuti dengan import berbagai library yang diperlukan untuk analisis data dan visualisasi. Library yang diimport meliputi Folium untuk visualisasi peta interaktif, Pandas untuk manipulasi data, NumPy untuk komputasi numerik, Matplotlib untuk plotting grafik, serta beberapa komponen dari mpl_toolkits untuk pengaturan sumbu dan plugin peta. Selain itu, terdapat import dari sklearn untuk clustering dengan DBSCAN dan preprocessing data menggunakan StandardScaler, yang mengindikasikan bahwa kode ini kemungkinan akan digunakan untuk analisis spasial atau clustering data geografis dengan visualisasi menggunakan peta.

1.2 Menghubungkan Google Colab dengan Google Drive

```
[217]
✓ 2 d
from google.colab import drive
drive.mount('/content/drive')

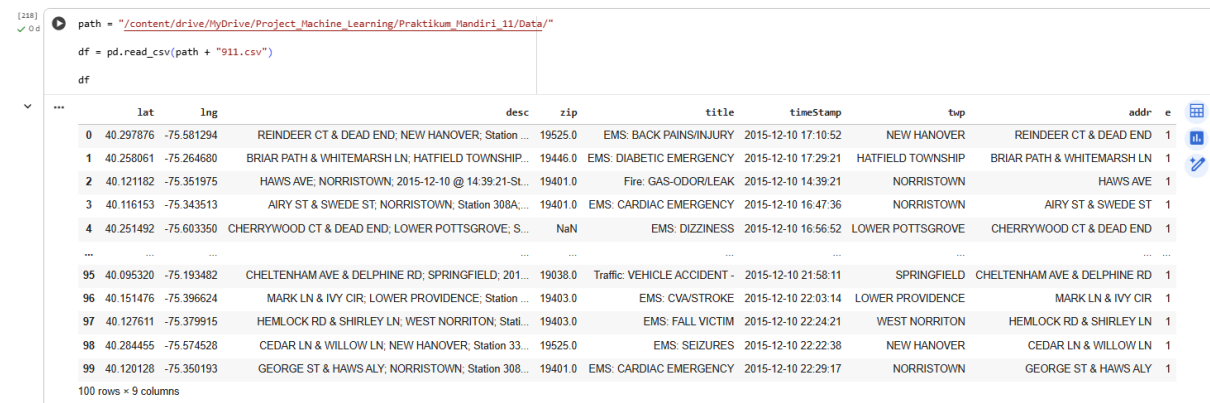
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Gambar 2. Menghubungkan Google Colab dengan Google Drive.

Kode ini berfungsi untuk menghubungkan atau mounting Google Drive ke dalam environment Google Colab agar dapat mengakses file-file yang tersimpan di Google Drive. Setelah menjalankan

kode ini, pengguna akan diminta untuk melakukan autentikasi dengan akun Google mereka, dan setelah berhasil, Google Drive akan terpasang pada direktori '/content/drive/'. Dengan demikian, semua file dan folder yang ada di Google Drive dapat diakses langsung dari notebook Colab untuk keperluan membaca dataset, menyimpan hasil analisis, atau operasi file lainnya tanpa perlu mengupload file secara manual setiap kali menjalankan notebook.

1.3 Membaca Dataset



```

path = "/content/drive/MyDrive/Project_Machine_Learning/Praktikum_Mandiri_11/Data/"

df = pd.read_csv(path + "911.csv")

df

```

	lat	lng	desc	zip	title	timeStamp	twp	addr	e
0	40.297876	-75.581294	REINDEER CT & DEAD END, NEW HANOVER, Station ...	19525.0	EMS: BACK PAINS/INJURY	2015-12-10 17:10:52	NEW HANOVER	REINDEER CT & DEAD END	1
1	40.258061	-75.264680	BRIAR PATH & WHITEMARSH LN, HATFIELD TOWNSHIP...	19446.0	EMS: DIABETIC EMERGENCY	2015-12-10 17:29:21	HATFIELD TOWNSHIP	BRIAR PATH & WHITEMARSH LN	1
2	40.121182	-75.351975	HAWS AVE, NORRISTOWN, 2015-12-10 @ 14:39:21-St...	19401.0	Fire: GAS-ODOR/LEAK	2015-12-10 14:39:21	NORRISTOWN	HAWS AVE	1
3	40.116153	-75.343513	AIRY ST & SWEDE ST, NORRISTOWN, Station 308A...	19401.0	EMS: CARDIAC EMERGENCY	2015-12-10 16:47:36	NORRISTOWN	AIRY ST & SWEDE ST	1
4	40.251492	-75.603350	CHERRYWOOD CT & DEAD END, LOWER POTTS GROVE, S...	NaN	EMS: DIZZINESS	2015-12-10 16:56:52	LOWER POTTS GROVE	CHERRYWOOD CT & DEAD END	1
...	...	...	...	...	...	...	...	...	...
95	40.095320	-75.193482	CHELTENHAM AVE & DELPHINE RD, SPRINGFIELD, 201...	19038.0	Traffic: VEHICLE ACCIDENT -	2015-12-10 21:58:11	SPRINGFIELD	CHELTENHAM AVE & DELPHINE RD	1
96	40.151476	-75.396624	MARK LN & IVY CIR, LOWER PROVIDENCE, Station ...	19403.0	EMS: CVA/STROKE	2015-12-10 22:03:14	LOWER PROVIDENCE	MARK LN & IVY CIR	1
97	40.127611	-75.379915	HEMLOCK RD & SHIRLEY LN, WEST NORRITON, Stati...	19403.0	EMS: FALL VICTIM	2015-12-10 22:24:21	WEST NORRITON	HEMLOCK RD & SHIRLEY LN	1
98	40.284455	-75.574528	CEDAR LN & WILLOW LN, NEW HANOVER, Station 33...	19525.0	EMS: SEIZURES	2015-12-10 22:22:38	NEW HANOVER	CEDAR LN & WILLOW LN	1
99	40.120128	-75.350193	GEORGE ST & HAWS ALY, NORRISTOWN, Station 308...	19401.0	EMS: CARDIAC EMERGENCY	2015-12-10 22:29:17	NORRISTOWN	GEORGE ST & HAWS ALY	1

100 rows x 9 columns

Gambar 3. Membaca Dataset.

Kode ini mendefinisikan path atau lokasi direktori tempat file data disimpan di Google Drive, kemudian membaca file CSV bernama "911.csv" menggunakan Pandas dan menyimpannya dalam variabel dataframe df. Path yang digunakan mengarah ke folder Project_Machine_Learning/Praktikum_Mandiri_11/Data di Google Drive pengguna, yang menunjukkan ini adalah bagian dari praktikum mandiri ke-11 dalam konteks pembelajaran machine learning. Dataset "911.csv" kemungkinan berisi data panggilan darurat 911 yang akan dianalisis lebih lanjut, dan dengan menampilkan df di akhir kode, pengguna dapat melihat preview data yang telah berhasil dimuat beserta struktur kolom dan beberapa baris datanya.

1.4 Pengecekan dan Pembersihan Data Missing Values

```
[219] df.isnull().sum()
✓ 0 d

0
lat    0
lng    0
desc    0
zip    14
title   0
timeStamp  0
twp     0
addr    0
e       0

dtype: int64

[220] df_clean = df.dropna(subset=["zip"])
✓ 0 d
```

Gambar 4. Pengecekan dan Pembersihan Data Missing Values.

Kode ini melakukan pengecekan nilai yang hilang (missing values) dalam dataset dengan menggunakan fungsi `isnull().sum()` yang akan menampilkan jumlah nilai null pada setiap kolom dataframe. Setelah mengidentifikasi missing values, dilakukan pembersihan data dengan menghapus baris-baris yang memiliki nilai null pada kolom "zip" menggunakan metode `dropna()`, dan hasilnya disimpan dalam dataframe baru bernama `df_clean`. Proses pembersihan ini penting dilakukan karena kolom zip (kode pos) kemungkinan akan digunakan untuk analisis geografis atau clustering spasial, sehingga data yang tidak memiliki informasi lokasi perlu dihapus agar tidak mengganggu akurasi analisis selanjutnya.

1.5 Pengecekan Data Duplikat

```
[221] df.duplicated().sum()
✓ 0 d

np.int64(0)
```

Gambar 5. Pengecekan Data Duplikat.

Kode ini berfungsi untuk memeriksa jumlah baris duplikat yang ada dalam dataset dengan menggunakan metode `duplicated().sum()`. Fungsi `duplicated()` akan mengidentifikasi setiap baris yang merupakan duplikasi sempurna dari baris lain dalam dataframe, kemudian `sum()` menghitung total jumlah baris duplikat tersebut. Pengecekan duplikasi data ini merupakan bagian penting dari tahap eksplorasi dan pembersihan data untuk memastikan integritas dataset, karena data duplikat dapat mempengaruhi hasil analisis statistik dan model machine learning dengan memberikan bobot berlebih pada observasi tertentu.

1.6 Ekstraksi Fitur GPS dan Clustering dengan DBSCAN

```
[222]
✓ 0 d

# Ambil kolom GPS
gps = df_clean[["lat", "lng"]].values

# Normalisasi data agar DBSCAN lebih stabil
scaled = StandardScaler().fit_transform(gps)

# Menentukan nilai eps dan min_samples (awal)
eps = 0.05      # bisa dituning
min_samples = 20

dbscan = DBSCAN(eps=eps, min_samples=min_samples).fit(scaled)
labels = dbscan.labels_

df_clean["cluster"] = labels
df_clean.to_csv("911_dbscan_result.csv", index=False)

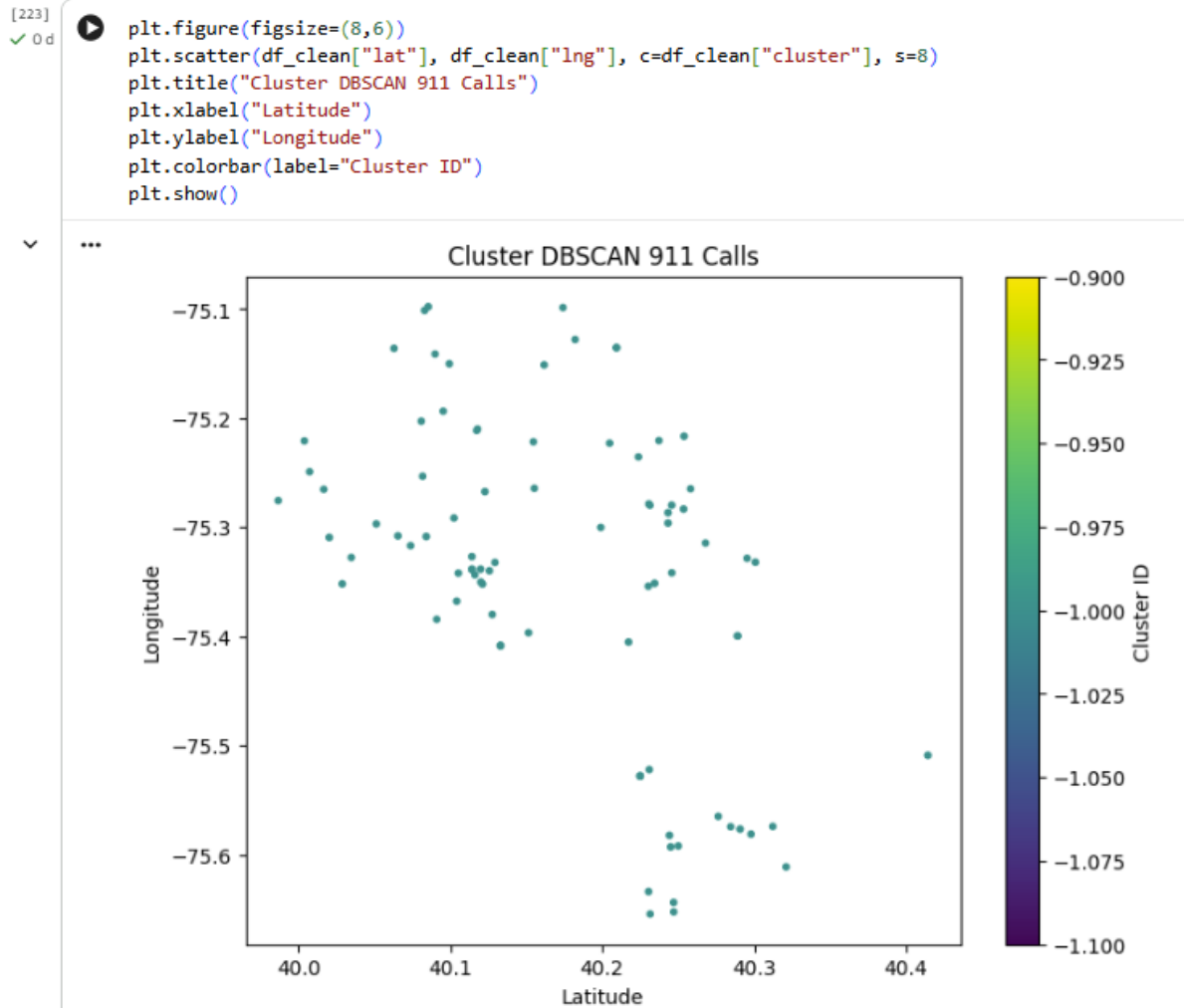
/tmp/ipython-input-2266166582.py:14: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df_clean["cluster"] = labels
```

Gambar 6. Ekstraksi Fitur GPS dan Clustering dengan DBSCAN.

Kode ini mengekstrak koordinat GPS (latitude dan longitude) dari dataframe yang telah dibersihkan dan menyimpannya dalam array NumPy bernama `gps`. Data koordinat tersebut kemudian dinormalisasi menggunakan `StandardScaler` agar memiliki skala yang seragam, yang penting untuk stabilitas algoritma DBSCAN karena algoritma ini sensitif terhadap skala data. Setelah normalisasi, dilakukan clustering menggunakan algoritma DBSCAN dengan parameter `eps` sebesar 0.05 yang menentukan jarak maksimum antar titik dalam satu cluster dan `min_samples` sebesar 20 yang menentukan jumlah minimum titik untuk membentuk cluster; kemudian hasil label cluster disimpan kembali ke kolom baru bernama "cluster" dalam `df_clean` dan diekspor ke file CSV dengan nama "911_dbscan_result.csv" untuk analisis lebih lanjut.

1.7 Visualisasi Hasil Clustering dengan Scatter Plot



Gambar 7. Visualisasi Hasil Clustering dengan Scatter Plot.

Kode ini membuat visualisasi scatter plot untuk menampilkan hasil clustering DBSCAN dari data panggilan darurat 911 berdasarkan koordinat geografisnya. Plot dibuat dengan ukuran 8x6 inci, dimana sumbu x merepresentasikan latitude, sumbu y merepresentasikan longitude, dan setiap titik diwarnai berdasarkan cluster ID-nya dengan ukuran marker sebesar 8 piksel. Visualisasi ini dilengkapi dengan judul "Cluster DBSCAN 911 Calls", label sumbu yang jelas, dan colorbar di samping grafik untuk menunjukkan skala warna yang merepresentasikan ID cluster berbeda, sehingga memudahkan interpretasi pola geografis dan distribusi spasial dari cluster-cluster panggilan darurat yang telah terbentuk.

1.8 Penentuan Parameter Epsilon dengan K-Distance Graph

[224]

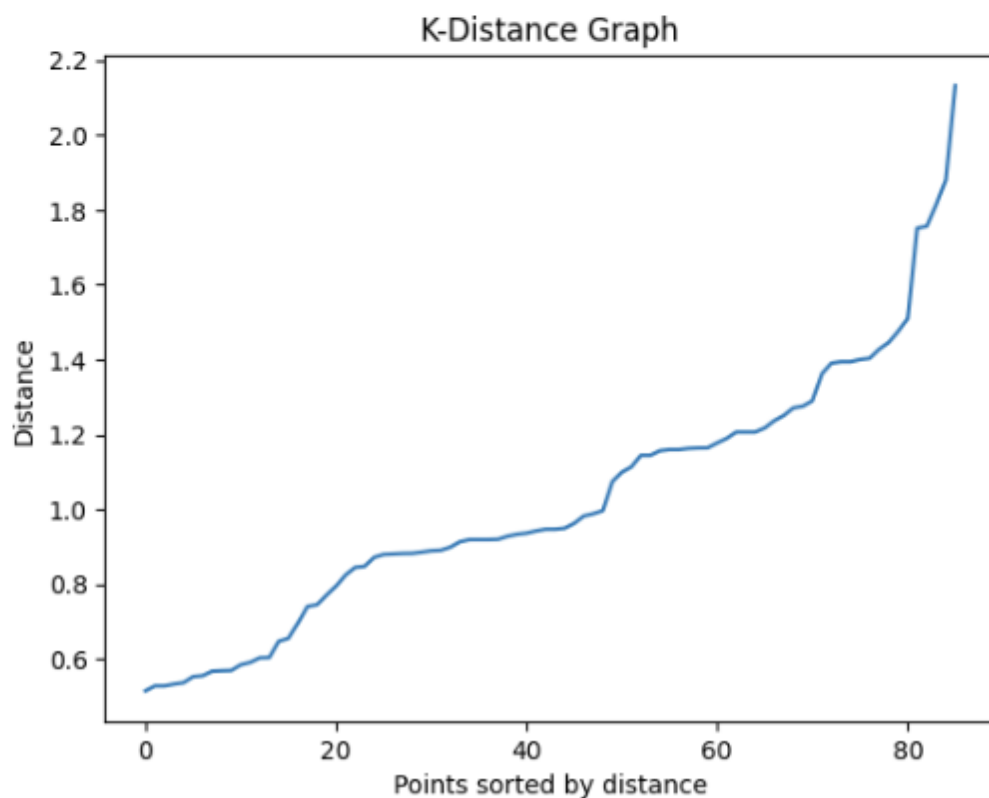
✓ 0 d

```
▶ k = 20
neigh = NearestNeighbors(n_neighbors=k)
nbrs = neigh.fit(scaled)
distances, indices = nbrs.kneighbors(scaled)

distances = np.sort(distances[:, k-1])
plt.plot(distances)
plt.title("K-Distance Graph")
plt.ylabel("Distance")
plt.xlabel("Points sorted by distance")
plt.show()
```

▼

...



Gambar 8. Penentuan Parameter Epsilon dengan K-Distance Graph.

Kode ini menggunakan algoritma K-Nearest Neighbors untuk membuat K-Distance Graph yang membantu menentukan nilai optimal parameter eps untuk DBSCAN. Pertama, algoritma mencari 20 tetangga terdekat untuk setiap titik data yang telah dinormalisasi, kemudian mengekstrak jarak ke tetangga ke-20 ($k-1$ karena index dimulai dari 0) untuk setiap titik dan mengurutkannya secara ascending. Hasil jarak tersebut divisualisasikan dalam bentuk grafik dimana sumbu x menunjukkan titik-titik data yang diurutkan berdasarkan jarak dan sumbu y menunjukkan nilai jarak, sehingga dengan mengidentifikasi titik "elbow" atau perubahan tajam pada kurva grafik ini, dapat ditentukan nilai eps yang optimal untuk memisahkan noise dari cluster yang sebenarnya dalam algoritma DBSCAN.

1.9 Analisis dan Evaluasi Hasil Clustering

```
[225]
✓ 0 d clusters = df_clean["cluster"].unique()
print("Daftar cluster:", clusters)
print("Jumlah Noise (-1):", sum(df_clean["cluster"] == -1))
print("\nJumlah titik tiap cluster:")
print(df_clean["cluster"].value_counts())

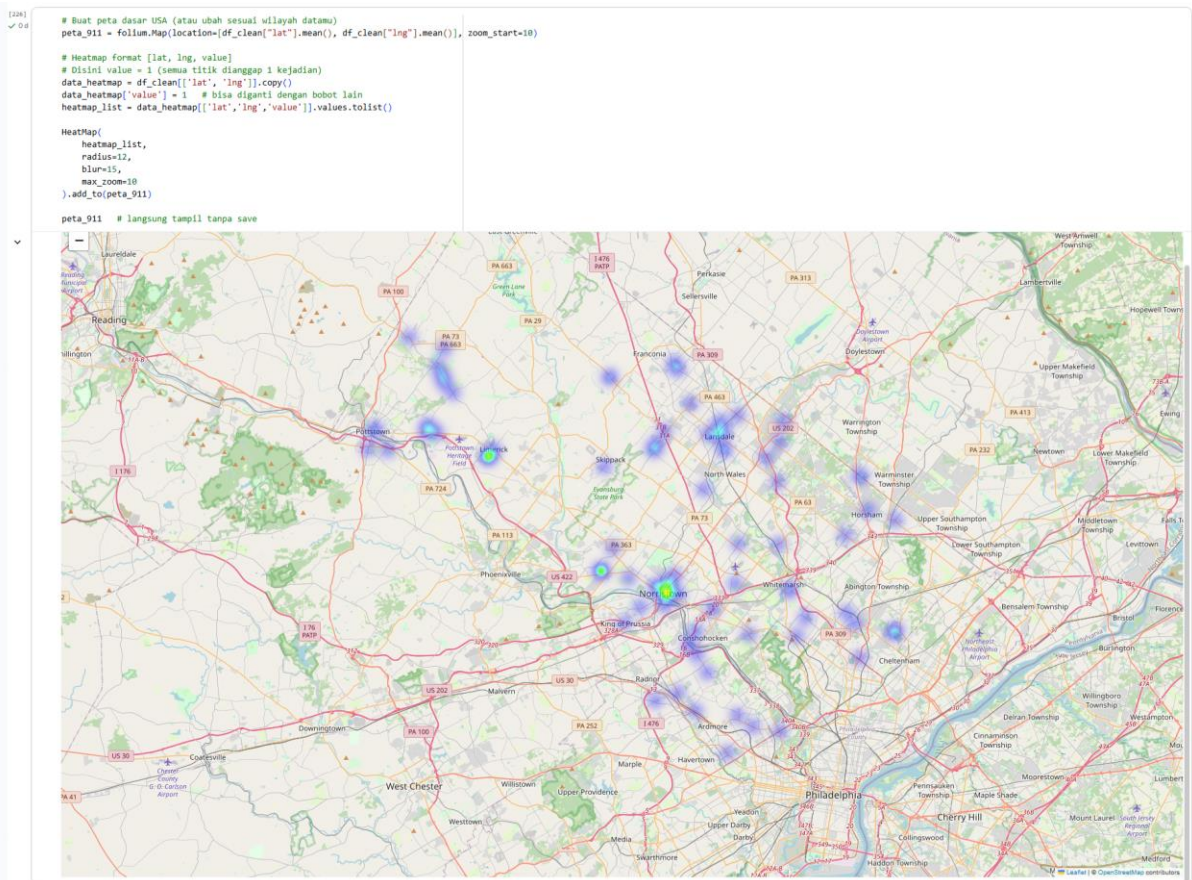
Daftar cluster: [-1]
Jumlah Noise (-1): 86

Jumlah titik tiap cluster:
cluster
-1      86
Name: count, dtype: int64
```

Gambar 9. Analisis dan Evaluasi Hasil Clustering.

Kode ini melakukan analisis terhadap hasil clustering DBSCAN dengan menampilkan informasi statistik tentang cluster yang terbentuk. Pertama, kode mengidentifikasi semua cluster ID yang unik dalam dataset dan menampilkannya, kemudian menghitung jumlah titik yang diklasifikasikan sebagai noise atau outlier yang ditandai dengan label -1 dalam DBSCAN. Selanjutnya, dilakukan penghitungan distribusi jumlah titik untuk setiap cluster menggunakan `value_counts()` yang akan menampilkan berapa banyak panggilan darurat 911 yang masuk ke dalam masing-masing cluster, sehingga memberikan gambaran tentang ukuran relatif setiap cluster dan membantu dalam mengevaluasi kualitas hasil clustering serta mengidentifikasi apakah ada cluster dominan atau distribusi yang tidak seimbang.

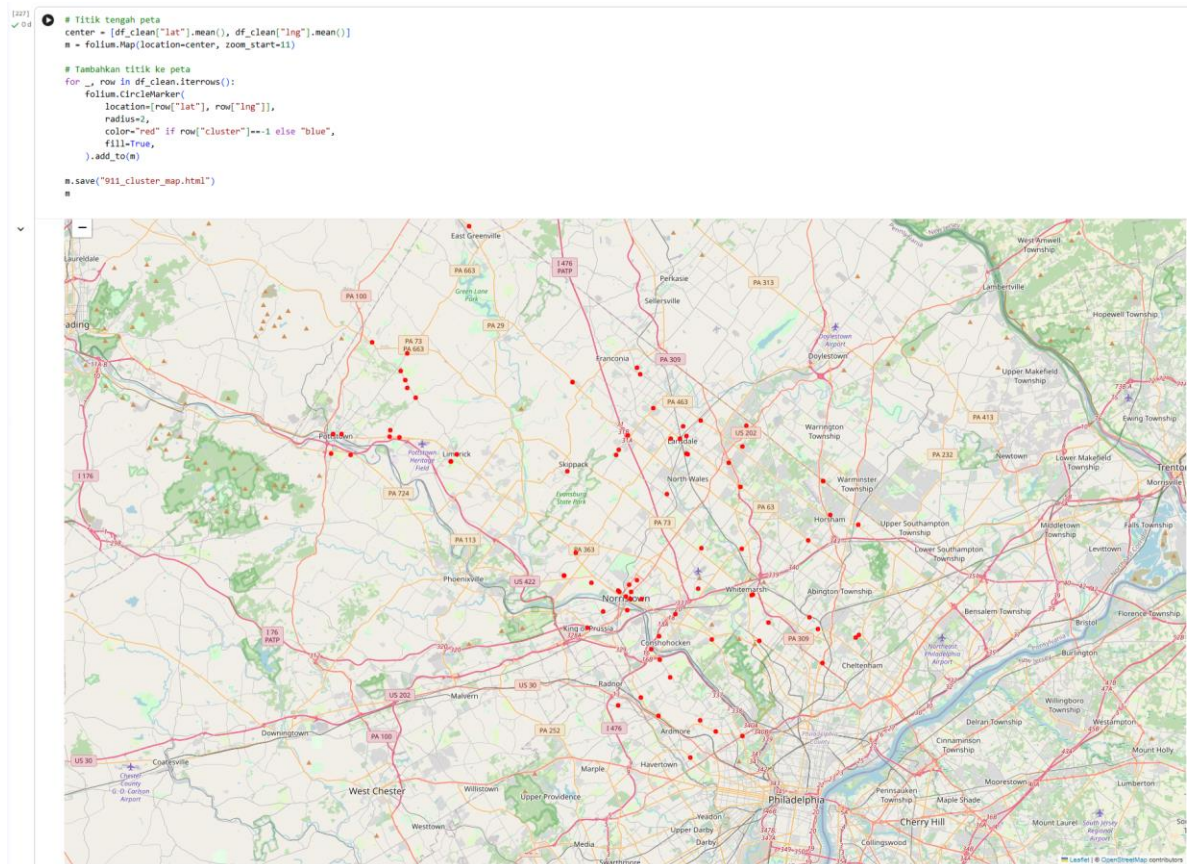
1.10 Visualisasi Heatmap Panggilan Darurat dengan Folium



Gambar 10. Visualisasi Heatmap Panggilan Darurat dengan Folium.

Kode ini membuat peta interaktif berbasis web menggunakan library Folium dengan heatmap untuk memvisualisasikan distribusi spasial panggilan darurat 911. Peta dasar dibuat dengan pusat lokasi berdasarkan rata-rata koordinat latitude dan longitude dari seluruh data, dengan tingkat zoom awal 10 untuk memberikan gambaran wilayah yang cukup luas. Data heatmap disiapkan dengan menambahkan kolom 'value' bernilai 1 untuk setiap titik yang merepresentasikan satu kejadian panggilan, kemudian dikonversi menjadi format list yang berisi koordinat [lat, lng, value] untuk diproses oleh HeatMap dengan parameter radius 12, blur 15, dan max_zoom 10 yang mengatur intensitas dan tampilan gradien panas pada peta. Hasil visualisasi ini memungkinkan identifikasi area-area dengan konsentrasi panggilan darurat tertinggi yang ditandai dengan warna lebih intens, sehingga berguna untuk analisis hotspot kejadian darurat di suatu wilayah.

1.11 Visualisasi Peta Interaktif Hasil Clustering dengan Marker



Gambar 11. Visualisasi Peta Interaktif Hasil Clustering dengan Marker.

Kode ini membuat peta interaktif yang menampilkan setiap titik panggilan darurat 911 sebagai marker individual dengan pewarnaan berdasarkan hasil clustering DBSCAN. Peta dibuat dengan titik pusat berdasarkan rata-rata koordinat latitude dan longitude dari data dengan zoom level 11, kemudian dilakukan iterasi untuk setiap baris data menggunakan `iterrows()` untuk menambahkan `CircleMarker` pada setiap lokasi panggilan. Setiap marker memiliki radius 2 piksel dan diwarnai merah jika teridentifikasi sebagai noise (`cluster -1`) atau biru jika termasuk dalam cluster tertentu, dengan properti `fill=True` untuk memberikan efek terisi penuh pada marker. Peta hasil visualisasi ini disimpan dalam file HTML bernama "911_cluster_map.html" yang dapat dibuka di browser untuk eksplorasi interaktif, memungkinkan pengguna untuk zoom in/out dan mengidentifikasi pola distribusi cluster serta outlier secara visual pada konteks geografis yang sebenarnya.